МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ПРОГРАММИРОВАНИЯ

КУРСОВАЯ РАБОТА
ЗАЩИЩЕНА С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

доц., к.т.н		А.В. Туманова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ

РАЗРАБОТКА ПРОГРАММЫ «КНИГОПОИСК»

по дисциплине: ОСНОВЫ ПРОГРАММИРОВАНИЯ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР.	4236		А. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

Содержание

1. ПОСТАНОВКА ЗАДАЧИ 3
1.1 ЗАДАЧА КУРСОВОЙ РАБОТЫ 3
1.2 ТИП ХРАНИМОЙ ИНФОРМАЦИИ И ЗАДАНИЕ 3
2. ОПИСАНИЕ СТРУКТУР ДАННЫХ 4
3. ОПИСАНИЕ ПРОГРАММЫ И СОЗДАННЫХ ФУНКЦИЙ 5
4. ОПИСАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА 8
5. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ ПРОГРАММЫ 8
ЗАКЛЮЧЕНИЕ 23
СПИСОК ЛИТЕРАТУРЫ 24
ПРИЛОЖЕНИЕ 1 26

1. Постановка задачи

1.1 Задача курсовой работы

Задачей курсовой работы является разработка программы для программы “Книгопоиск” с использованием заданных структур данных, которая позволяет вводить информацию, хранить её в базе данных SQL, осуществлять поиск, редактирование, добавление и удаление данных.

1.2 Тип хранимой информации и задание

Данные о книгах хранятся в структуре Book, содержащей следующие поля:

- номер каждой книги в базе данных;
- название книг;
- автор;
- год выпуска;
- жанр.

Задание на поиск: найти книги по запросу пользователя и дать ему возможность добавлять, удалять и редактировать книги в базе данных с помощью интерфейса в виде cmd.

2. Описание структур данных

Таблица базы данных предоставлена в таблице 1:

Поле	Тип
id	int
title	varchar(255)
author	varchar(255)
year	char(10)
gendre	varchar(255)

Таблица 1 – База данных

Данные о книгах хранятся в структуре Book

```
struct Book {
    int id; //Номер книги в базе данных
    std::string title; //Название книги
    std::string author; //Автор
    std::string year; //Год выпуска
    std::string genre; //Жанр
};
```

Для хранения данных используется SQL база данных. Данные хранятся в таблицах с соответствующей структурой. В процессе выполнения программы данные из базы данных извлекаются и обрабатываются. Номера книг уникальны и используются для идентификации записей. Информация о допустимых типах книг хранится в отдельной таблице или таблицах в базе данных.

3. Описание программы и созданных функций

Программа реализована на языке C++ в виде консольного приложения. В главной функции main() реализовано меню пользователя, в котором каждому действию соответствует определенная цифра. Реализованы следующие функции для работы с данными: проверка на вводимое время, поиск по названию, поиск по автору, поиск по жанру, добавление, редактирование, удаление книги, вывод всех книг из базы данных, проверка действительного времени.

Поиск по названию, автору и жанру

searchBooks(int searchOption)

Функция выполняет поиск книги по названию, автору и жанру в зависимости от выбора пользователя. При вводе данных выполняется проверка есть ли данная книга с таким названием, автором или жанром в базе данных. Если введены некорректные данные, пользователю выводит сообщение о том, что такой книги в базе данных нет.

Добавление книги

void addBook()

Функция выполняет добавление книги в базу данных. При вводе названия книги выполняется проверка, есть ли такая книга с таким же названием в базе данных, если есть, то выводит сообщение пользователю о том что такая книга уже есть и нужно ввести повторно. При добавлении так же пользователь вводит автора книги, жанр и год выпуска. При вводе года выпуска выполняется проверка, если это некорректное время то просит ввести заново по формату xx.xx.xxxx.

Редактирование книги

void editBook()

Функция редактирования книги даёт пользователю редактировать название книги, автора, жанр или год выпуска. Внутри этой функции так же даётся выбор, редактировать полностью книгу или только какую-то часть этой книги.

Удаление книги

void deleteBook()

Функция удаления книги даёт пользователю удалить книгу по названию. Если пользователь вводит книгу, которой нет в базе данных, то пользователю выводит сообщение о том, что такой книги для удаления из базы данных нету.

Вывод всех книг

void printAllBooks()

Функция вывода всех книг из базы данных даёт пользователю увидеть все книги внутри базы данных. Эта функция выводит все книги с их названиями, авторами, годом выпуска и жанром в cmd.

Удаление всей БД

void clearDatabase()

Она выполняет SQL-запрос DELETE FROM books;, который удаляет все записи из таблицы "books". Если операция успешно завершена, она выводит сообщение "База данных успешно очищена." В противном случае, если произошла ошибка при выполнении запроса, она выводит сообщение об ошибке.

Преобразование первой буквы текста в заглавную

void toUpper(std::string& text)

Преобразует первую букву в строке text в верхний регистр, если она является латинской (a-z) или кириллической (а-я) буквой. Если строка пуста или первый символ не является буквой, функция ничего не делает.

Проверка на ввод с заглавной буквы

void isFirstLetterCapitalAndLatin(const std::string& input)

Функция проверяет, начинается ли переданная строка с буквы, которая является заглавной и принадлежит латинскому или кириллическому алфавиту. Если это условие выполняется, функция возвращает true, в противном случае возвращает false.

Проверка действительного времени

void isValidYear(const std::string& data)

Функция проверка действительного времени проверяет вводимый год выпуска для книги. Она сопоставляет вводимое время пользователя с текущим временем, что оно не может превышать его и реальность месяца, дня и года по формату xx.xx.xxxx.

Сортировка и вывод списка книг

printBooksSortedByAll(const std::string& criteria)

Функция вывода отсортированного списка книг из базы данных даёт пользователю увидеть все книги с разными способами сортировки. Эта функция выводит все книги с их названиями, авторами, годом выпуска и жанром на экран.

Полный код предоставлен в Приложении 1.

4. Описание пользовательского интерфейса

При запуске программы на экран выводится консольное приложение с меню пользователя. При нажатии на клавиатуре на определенную цифру выполняется соответствующая функция.

1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти

5. Результаты тестирования программы

Тестирование выполняется для программы целиком, показывает корректное взаимодействие всех функций системы.

Приводится набор тестов (ручное тестирование), на которых была проверена работа спроектированной программы.

В отчете результаты тестирования приводятся в виде скриншотов.

Попав в главное меню, мы проверили есть ли проверка на ввод не числа и числа которого нет в пункте меню (рис.1).

```
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: a

Такого числа в пункте меню нет.
Выберите повторно: -1

Такого числа в пункте меню нет.
```

Рисунок 1 – Проверка на ввод

При выборе первого пункта меню, мы пишем название книги. Введя книгу, которой нет в базе данных выводит сообщение о том, что такой книги нет, но при вводе книги, которая есть не важно с заглавной или строчной буквы, то оно её найдет (рис.2).

```
Введите название книги для поиска: hoika
Номер              Название              Автор                 Дата                  Жанр
-------------------------------------------------------------------------------------------------
1                  Hoika                 Dima                  01.02.2000            Fantasy
Введите название книги для поиска: fsa
Книги не найдены.
```

Рисунок 2 – Выбрали поиск книги по названию

При выборе второго пункта меню мы вводим автора книг. Введя автора, которого нету в базе данных программа сообщает о том, что книг такого автора нет, а если есть то выведет все его кинги. Так же можно писать как с строчной, так и с заглавной буквы (рис.3).

```
Введите число: 2

Введите имя автора для поиска книг: fas
Книги не найдены.
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: 2

Введите имя автора для поиска книг: Dima
Номер               Название               Автор                  Дата                   Жанр
-----------------------------------------------------------------------------------------------------
1                   Hoika                  Dima                   01.02.2000             Fantasy
```

Рисунок 3 – Выбрали поиск книги по автору

При выборе третьего пункта меню мы вводим жанр книг. Введя жанр, которого нету в базе данных программа сообщает о том, что книг такого жанра нет, а если есть то выведет все книги данного жанра. Так же можно писать как с строчной, так и с заглавной буквы (рис.4).

```
Введите число: 3

Введите жанр книги для поиска: fas
Книги не найдены.
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: 3

Введите жанр книги для поиска: Fantasy
Номер               Название               Автор                  Дата                   Жанр
-----------------------------------------------------------------------------------------------------
1                   Hoika                  Dima                   01.02.2000             Fantasy
```

Рисунок 4 – Выбрали поиск книги по жанру

При выборе четвертого пункта меню мы вводим новую книгу. Сначала просит ввести название книги, которое обязательно должно быть с заглавной

буквы либо цифры, далее просит ввести автора так же обязательно с заглавной буквы либо цифры далее проверяется формат даты, введенный пользователем и если он некорректен, то просит написать заново ну и в конце жанр, который так же должен быть с заглавной буквы либо с цифры. Если книга с таким названием уже есть в базе данных то просит ввести заново книгу. (рис.5).

```
Введите число: 4

Введите название книги: gas
Название книги должно начинаться с заглавной буквы либо с цифры.
Введите название книги: Gaz
Введите автора книги: hfkd
Автор должен начинаться с заглавной буквы либо с цифры.
Введите автора книги: Fedor
Введите дату выпуска (в формате xx.xx.xxxx): gas
Неверный формат даты. Пожалуйста, введите дату в формате xx.xx.xxxx.
Введите дату выпуска (в формате xx.xx.xxxx): 532
Неверный формат даты. Пожалуйста, введите дату в формате xx.xx.xxxx.
Введите дату выпуска (в формате xx.xx.xxxx): 01.09.2019
Введите жанр книги: fas
Жанр должен начинаться с заглавной буквы либо с цифры.
Введите жанр книги: Lop
Книга успешно добавлена!
```

Рисунок 5 – Выбрали добавление книги

При выборе пятого пункта меню редактируем книгу по выбору. Если книги для редактирования нет которую вписал пользователь об этом ему сообщается что такой книги нет. Так же можно вводить поиск книги с строчной буквы и нас перекидывает в пункт меню редактирования (рис.6).

```
Введите название книги для редактирования: fas
Книга с названием 'Fas' не найдена в базе данных.
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: 5

Введите название книги для редактирования: hoika
Выберите, что вы хотите отредактировать:
1. Полное редактирование книги
2. Редактировать только название книги
3. Редактировать только автора книги
4. Редактировать только жанр книги
5. Редактировать только дату выпуска книги
0. Отменить редактирование и вернуться в главное меню
Введите число: |
```

Рисунок 6 – Выбрали редактирование книги

При выборе первого пункта меню редактирования мы вводим новое название, автора, жанр и год выпуска со всеми проверками. После чего нас перекидывает в главное меню. Если при редактирование вписали название книги которое есть в базе данных то нам об этом сообщается, и книга не редактируется (рис.7).

```
Введите число: 1
Введите новое название книги: gsaa
Название книги должно начинаться с заглавной буквы либо с цифры.
Введите новое название книги: Gaz
Книга 'Gaz' уже существует в базе данных. Пожалуйста, введите другое название.
Введите новое название книги: Kol
Введите нового автора книги: fsa
Автор должен начинаться с заглавной буквы либо с цифры.
Введите нового автора книги: Chelovek
Введите новый дату выпуска (в формате xx.xx.xxxx): 42
Неверный формат даты. Пожалуйста, введите дату в формате xx.xx.xxxx.
Введите новый дату выпуска (в формате xx.xx.xxxx): fsa
Неверный формат даты. Пожалуйста, введите дату в формате xx.xx.xxxx.
Введите новый дату выпуска (в формате xx.xx.xxxx): 19.12.2004
Введите новый жанр книги: saf
Жанр должен начинаться с заглавной буквы либо с цифры.
Введите новый жанр книги: Hol
Книга успешно отредактирована!
```

Рисунок 7 – Выбрали первый пункт меню редактирования

При выборе второго пункта меню редактирования мы вводим новое название книги, но если книга при редактировании уже есть с таким названием что и в базе данных то книга не от редактируется и пользователю об этом сообщится. После чего нас перекидывает в главное меню (рис.8).

```
Введите число: 2
Введите новое название книги: Gaz
Книга 'Gaz' уже существует в базе данных. Пожалуйста, введите другое название.
Введите новое название книги: Hoi
Название книги успешно обновлено!
```

Рисунок 8 – Выбрали второй пункт меню редактирования

При выборе третьего пункта меню редактирования мы вводим нового автора книги. После чего нас перекидывает в меню редактирования (рис.9).

```
Введите число: 3
Введите нового автора книги: gleb
Автор должен начинаться с заглавной буквы либо с цифры.
Введите нового автора книги: Gleb
Автор книги успешно отредактирован!
```

Рисунок 9 – Выбрали третий пункт меню редактирования

При выборе четвертого пункта меню редактирования мы вводим новый жанр книги. После чего нас переносит обратно в меню редактирования (рис.10).

```
Введите число: 4
Введите новый жанр книги: fas
Жанр должен начинаться с заглавной буквы либо с цифры.
Введите новый жанр книги: Lodes
Жанр книги успешно отредактирован!
```

Рисунок 10 – Выбрали четвёртый пункт меню редактирования

При выборе пятого пункта меню редактирования мы вводим новую дату выпуска книги. При полной проверка на действительность времени нас переносит обратно в меню редактирования (рис.11).

```
Введите число: 5
Введите новую дату выпуска (в формате xx.xx.xxxx): sfa
Неверный формат даты. Пожалуйста, введите дату в формате xx.xx.xxxx.
Введите новую дату выпуска (в формате xx.xx.xxxx): 421
Неверный формат даты. Пожалуйста, введите дату в формате xx.xx.xxxx.
Введите новую дату выпуска (в формате xx.xx.xxxx): 10.10.1999
Дата выпуска книги успешно отредактирована!
```

Рисунок 11 – Выбрали пятый пункт меню редактирования

При выборе нулевого пункта меню нас обратно переносит в главное меню (рис.12).

```
Введите число: 0
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число:
```

Рисунок 12 – Выбрали пункт ноль меню редактирования

При выборе шестого пункта главного меню мы вводим название книги для удаления. Если такой книги нет, то программа пишет, что такой книги в базе данных нет. Так же можно писать не с заглавной буквы для того, чтобы найти книгу для удаления (рис.13).

```
Введите название книги для удаления: gasgasf
Книги с таким названием не найдены.
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: 6

Введите название книги для удаления: hoi
Книга успешно удалена!
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: 5
```

Рисунок 13 – Выбрали удаление книги

При выборе седьмого пункта главного меню нам красиво отображает содержимое базы данных о книгах (рис.14).

```
Введите число: 7

Номер               Название               Автор                  Дата                   Жанр
-------------------------------------------------------------------------------------------------
1                   Gaz                    Fedor                  01.09.2019             Lop
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: |
```

Рисунок 14 – Выбрали вывод всех книг

При выборе восьмого пункта главного меню мы попадаем в новое меню с сортировками выберем первый пункт (рис.15).

```
Введите число: 8

1. Список книг отсортированных по названию
2. Список книг отсортированных по автору
3. Список книг отсортированных по дате выпуска
4. Список книг отсортированных по жанру
5. Выход в главное меню
Введите число: |
```

Рисунок 15 – Выбрали отсортировку книг

При выборе первого пункта, нам выводит список отсортированных книг по названию (рис.16).

```
Номер              Название              Автор                 Дата                  Жанр
-----------------------------------------------------------------------------------------------
1                  Arbyz                 Andrey                20.04.1900            Plov
2                  Gaz                   Fedor                 01.09.2019            Lop
3                  Koi                   Iok                   04.02.2022            Gde

1. Список книг отсортированных по названию
2. Список книг отсортированных по автору
3. Список книг отсортированных по дате выпуска
4. Список книг отсортированных по жанру
5. Выход в главное меню
Введите число: |
```

Рисунок 16 – Выбрали первый пункт подменю сортировки

При выборе второго пункта, нам выводит список отсортированных книг по автору (рис.17).

```
Номер              Название              Автор                 Дата                  Жанр
-----------------------------------------------------------------------------------------------
1                  Arbyz                 Andrey                20.04.1900            Plov
2                  Gaz                   Fedor                 01.09.2019            Lop
3                  Koi                   Iok                   04.02.2022            Gde

1. Список книг отсортированных по названию
2. Список книг отсортированных по автору
3. Список книг отсортированных по дате выпуска
4. Список книг отсортированных по жанру
5. Выход в главное меню
Введите число: |
```

Рисунок 17 – Выбрали второй пункт подменю сортировки

При выборе третьего пункта, нам выводит список отсортированных книг по дате выпуска (рис.18).

```
Номер              Название              Автор                 Дата                  Жанр
-----------------------------------------------------------------------------------------------
1                  Koi                   Iok                   04.02.2022            Gde
2                  Gaz                   Fedor                 01.09.2019            Lop
3                  Arbyz                 Andrey                20.04.1900            Plov

1. Список книг отсортированных по названию
2. Список книг отсортированных по автору
3. Список книг отсортированных по дате выпуска
4. Список книг отсортированных по жанру
5. Выход в главное меню
Введите число: |
```

Рисунок 18 – Выбрали третий пункт подменю сортировки

При выборе четвертого пункта, нам выводит список отсортированных книг по жанру (рис.19).

```
Введите число: 4

Номер               Название               Автор                  Дата                   Жанр
-----------------------------------------------------------------------------------------------
1                   Koi                    Iok                    04.02.2022             Gde
2                   Gaz                    Fedor                  01.09.2019             Lop
3                   Arbyz                  Andrey                 20.04.1900             Plov

1. Список книг отсортированных по названию
2. Список книг отсортированных по автору
3. Список книг отсортированных по дате выпуска
4. Список книг отсортированных по жанру
5. Выход в главное меню
Введите число:
```

Рисунок 19 - Выбрали четвертый пункт подменю сортировки

Вписав число или не число которого нету в подменю сортировки мы видим, что нам сообщают о том, что такого нет и просят ввести заново (рис.20).

```
Введите число: афы

Такого числа в пункте меню нет.
Выберите повторно: 0

Такого числа в пункте меню нет.
Выберите повторно:
```

Рисунок 20 – Проверка ввода в подменю сортировки

При выборе пятого пункта мы видим, что нас вернуло обратно в главное меню программы (рис.21).

```
Выберите повторно: 5
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число:
```

Рисунок 21 – Выбрали пятый пункт подменю сортировки

При выборе девятого пункта главного меню, мы удаляем все книги из базы данных. Выбрав снова седьмой пункт меню, мы видим, что книг в нем после девятого пункта нет (рис.22).

```
Введите число: 9

База данных успешно очищена.
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: 7

Книги не найдены.
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: ы
```

Рисунок 22 – Выбрали чистку базы данных

При выборе нулевого пункта главного меню мы можем выйти из программы (рис.23).

```
Выберите операцию:
1. Поиск книги по названию
2. Поиск книги по автору
3. Поиск книги по жанру
4. Добавить книгу
5. Редактировать книгу
6. Удалить книгу
7. Вывести все книги из базы данных
8. Вывод отсортированных книг
9. Очистить базу данных
0. Выйти
Введите число: 0

Программа завершена.
```

Рисунок 23 – Выбрали выход из программы

Заключение

Приведенное приложение представляет собой консольное приложение для управления базой данных книг. Он использует библиотеку SQLite3 для создания и управления базой данных. В приложении реализованы следующие функции:

1. Добавление новых книг в базу данных, с проверкой на уникальность названия книги.
2. Редактирование информации о книге, включая изменение названия, автора, года выпуска и жанра.
3. Удаление книги из базы данных по названию.
4. Поиск книг по названию, автору и жанру.
5. Вывод всех книг из базы данных.
6. Вывод отсортированных книг
7. Удаление всей базы данных

Приложение также обеспечивает проверку на правильность формата года выпуска и корректный ввод данных пользователем.

Этот код может быть полезен для создания простой библиотечной системы или другой системы управления данными. Он демонстрирует применение SQLite для хранения и манипулирования данными, а также использование структур данных и функций для взаимодействия с базой данных.

Список использованных источников

1. SQLite3 документация — официальная документация SQLite3 для работы с базами данных, URL: https://www.sqlite.org/docs.html (дата обращения 04.10.2023).
2. Документация по стандартной библиотеке C++ — ресурс для изучения стандартной библиотеки C++, URL: https://en.cppreference.com/w/ (дата обращения 04.10.2023).
3. CPlusPlus.com — сайт для получения информации о C++ и его библиотеках, URL: https://www.cplusplus.com/ (дата обращения 04.10.2023).
4. Статья на википедии о регулярных выражениях — информация о работе с регулярными выражениями в C++, URL: https://ru.wikipedia.org/wiki/%D0%A0%D0%B5%D0%B3%D1%83%D0%BB%D1%8F%D1%80%D0%BD%D1%8B%D0%B5_%D0%B2%D1%8B%D1%80%D0%B0%D0%B6%D0%B5%D0%BD%D0%B8%D1%8F (дата обращения 04.10.2023).
5. Microsoft Docs — документация по функции GetLocalTime из Windows API, URL: https://docs.microsoft.com/enus/windows/win32/api/sysinfoapi/nf-sysinfoapi-getlocaltime (дата обращения 04.10.2023).
6. Статья на сайте CPlusPlus.com о работе с SQLite в C++, URL: https://www.cplusplus.com/forum/windows/272553/ (дата обращения 04.10.2023).
7. Статья на сайте dev.to о работе с SQLite и C++, URL: https://dev.to/telmo/how-to-use-sqlite-3-with-c-2am2 (дата обращения 04.10.2023).
8. Статья на сайте Stack Overflow о подготовке и выполнении SQL-запросов в C++ с использованием SQLite3, URL: https://stackoverflow.com/questio

ns/24522639/how-to-prepare-and-execute-sql-queries-in-c-using-sqlite3 (дата обращения 04.10.2023).

9. Статья на сайте Cprogramming.com о работе с SQLite и C++, URL: https://www.cprogramming.com/tutorial/sqlite/intro.html (дата обращения 04.10.2023).
10. Статья на сайте Cpp.sh о работе с регулярными выражениями в C++, URL: http://cpp.sh/ (дата обращения 04.10.2023).

ПРИЛОЖЕНИЕ 1

Текст программы

```
#include <iostream>
#include <sqlite3.h>
#include <string>
#include <regex>
#include <ctime>
#include <windows.h>
#include <cctype>
#include <locale>
#include <iomanip>

struct Book {
    int id;
    std::string title;
    std::string author;
    std::string data;
    std::string genre;
};

sqlite3* db = nullptr;

int executeSQL(const std::string& query, int (*callback)(void*, int, char**, char**),
void* data = nullptr, char** errMsg = nullptr) {
    return sqlite3_exec(db, query.c_str(), callback, data, errMsg);
}

void toUpper(std::string& text) {
    if (text.empty()) {
        return;
    }

    char& firstChar = text[0];
    if ((firstChar >= 'a' && firstChar <= 'z') || (firstChar >= 'а' && firstChar <=
'я')) {
        // Преобразование латинской буквы 'a'-'z' или кириллической буквы 'а'-'я' в
верхний регистр
        if (firstChar >= 'a' && firstChar <= 'z') {
            firstChar -= 'a' - 'A'; // Для латиницы
        }
        else {
            firstChar -= 'а' - 'А'; // Для кириллицы
        }
    }
}

bool isFirstLetterCapitalAndLatin(const std::string& input) {
    if (!input.empty()) {
        char firstChar = input[0];
        // Добавим проверку на цифры (0-9) и пропустим, если первый символ цифра.
        if (firstChar >= '0' && firstChar <= '9') {
            return true;
        }
        if ((firstChar >= 'A' && firstChar <= 'Z') || (firstChar >= 'А' && firstChar <=
'Я')) {
            return true;
        }
    }
```

```
    return false;
}

bool isValidYear(const std::string& data) {
    std::regex yearPattern(R"(^\d{2}\.\d{2}\.\d{4}$)");
    if (!std::regex_match(data, yearPattern)) {
        return false;
    }

    SYSTEMTIME sysTime;
    GetLocalTime(&sysTime);

    int currentYear = sysTime.wYear;

    int bookYear = std::stoi(data.substr(6, 4));
    int bookMonth = std::stoi(data.substr(3, 2));
    int bookDay = std::stoi(data.substr(0, 2));

    if (bookMonth < 1 || bookMonth > 12 || bookDay < 1) {
        return false;
    }

    if (bookMonth == 2) {
        bool isLeapYear = (bookYear % 4 == 0 && bookYear % 100 != 0) || (bookYear % 400
== 0);
        if (isLeapYear) {
            if (bookDay > 29) {
                return false;
            }
        }
        else {
            if (bookDay > 28) {
                return false;
            }
        }
    }
    else if ((bookMonth == 4 || bookMonth == 6 || bookMonth == 9 || bookMonth == 11) &&
bookDay > 30) {
        return false;
    }
    else if (bookDay > 31) {
        return false;
    }

    if (bookYear > currentYear) {
        return false;
    }

    return true;
}

void addBook() {
    Book book;
    std::cin.ignore();

    do {
        while (true) {
            std::cout << "Введите название книги: ";
            std::getline(std::cin, book.title);
            if (isFirstLetterCapitalAndLatin(book.title)) {
```

```
                break;
            }
            else {
                std::cout << "Название книги должно начинаться с заглавной буквы либо с
цифры." << std::endl;
            }
        }

        // Для проверки наличия книги с таким названием
        std::string checkBookSQL = "SELECT COUNT(*) FROM books WHERE title = '" +
book.title + "';";
        char* errMsg = nullptr;
        sqlite3_stmt* stmt;

        int rc = sqlite3_prepare_v2(db, checkBookSQL.c_str(), -1, &stmt, nullptr);
        if (rc != SQLITE_OK) {
            std::cerr << "Ошибка подготовки SQL-запроса: " << sqlite3_errmsg(db) <<
std::endl;
            return;
        }

        rc = sqlite3_step(stmt);
        if (rc == SQLITE_ROW) {
            int count = sqlite3_column_int(stmt, 0);
            if (count > 0) {
                std::cout << "Книга с таким названием уже существует. Пожалуйста,
введите название книги заново.\n";
            }
            else {
                break;
            }
        }

        sqlite3_finalize(stmt);
    } while (true);

    while (true) {
        std::cout << "Введите автора книги: ";
        std::getline(std::cin, book.author);
        if (isFirstLetterCapitalAndLatin(book.author)) {
            break;
        }
        else {
            std::cout << "Автор должен начинаться с заглавной буквы либо с цифры." <<
std::endl;
        }
    }

    do {
        std::cout << "Введите дату выпуска (в формате xx.xx.xxxx): ";
        std::getline(std::cin, book.data);
        if (!isValidYear(book.data)) {
            std::cout << "Неверный формат даты. Пожалуйста, введите дату в формате
xx.xx.xxxx.\n";
        }
    } while (!isValidYear(book.data));

    while (true) {
        std::cout << "Введите жанр книги: ";
        std::getline(std::cin, book.genre);
```

```
        if (isFirstLetterCapitalAndLatin(book.genre)) {
            break;
        }
        else {
            std::cout << "Жанр должен начинаться с заглавной буквы либо с цифры." <<
std::endl;
        }
    }

    std::string addBookSQL = "INSERT INTO books (title, author, data, genre) VALUES ('"
+
        book.title + "', '" + book.author + "', '" + book.data + "', '" + book.genre +
"');";
    char* errMsg = nullptr;
    int rc = executeSQL(addBookSQL, 0, 0, &errMsg);

    if (rc != SQLITE_OK) {
        std::cerr << "Ошибка добавления книги: " << errMsg << std::endl;
        sqlite3_free(errMsg);
    }
    else {
        std::cout << "Книга успешно добавлена!\n";
    }
}

void deleteBook() {
    std::cin.ignore();
    std::cout << "Введите название книги для удаления: ";
    std::string bookTitle;
    std::getline(std::cin, bookTitle);

    toUpper(bookTitle);

    std::string deleteBookSQL = "DELETE FROM books WHERE title = '" + bookTitle + "';";
    char* errMsg = nullptr;
    int rc = executeSQL(deleteBookSQL, 0, 0, &errMsg);

    if (rc != SQLITE_OK) {
        std::cerr << "Ошибка удаления книги: " << errMsg << std::endl;
        sqlite3_free(errMsg);
    }
    else {
        if (sqlite3_changes(db) == 0) {
            std::cout << "Книги с таким названием не найдены.\n";
        }
        else {
            std::cout << "Книга успешно удалена!\n";
        }
    }
}

void editBook() {
    std::cin.ignore();
    std::cout << "Введите название книги для редактирования: ";
    std::string bookTitle;
    std::getline(std::cin, bookTitle);

    toUpper(bookTitle);
```

```
    std::string selectBookSQL = "SELECT * FROM books WHERE title = '" + bookTitle +
"';";
    char* errMsg = nullptr;
    sqlite3_stmt* stmt;

    int rc = sqlite3_prepare_v2(db, selectBookSQL.c_str(), -1, &stmt, nullptr);

    if (rc != SQLITE_OK) {
        std::cerr << "Ошибка подготовки SQL-запроса: " << sqlite3_errmsg(db) <<
std::endl;
        sqlite3_finalize(stmt);
        return;
    }

    rc = sqlite3_step(stmt);

    if (rc != SQLITE_ROW) {
        std::cout << "Книга с названием '" << bookTitle << "' не найдена в базе
данных." << std::endl;
        sqlite3_finalize(stmt);
        return;
    }
    else {
        int editChoice;
        do {
            std::cout << "Выберите, что вы хотите отредактировать:\n"
                << "1. Полное редактирование книги\n"
                << "2. Редактировать только название книги\n"
                << "3. Редактировать только автора книги\n"
                << "4. Редактировать только жанр книги\n"
                << "5. Редактировать только дату выпуска книги\n"
                << "0. Отменить редактирование и вернуться в главное меню\n"
                << "Введите число: ";

            std::cin >> editChoice;
            while (std::cin.fail())
            {
                std::cin.clear();
                std::cin.ignore(10000, '\n');
                std::cout << "\n";
                std::cout << "Такого числа в пункте меню нет.\n";
                std::cout << "Введите повторно: ";
                std::cin >> editChoice;
            }

            // Очистить буфер после ввода числа
            std::cin.ignore(10000, '\n');

            switch (editChoice) {
            case 1:
            {
                Book book;
                bool isUnique = false;

                while (!isUnique) {
                    while (true) {
                        std::cout << "Введите новое название книги: ";
                        std::getline(std::cin, book.title);
                        if (isFirstLetterCapitalAndLatin(book.title)) {
                            break;
```

```
                        }
                        else {
                            std::cout << "Название книги должно начинаться с заглавной
буквы либо с цифры." << std::endl;
                        }
                    }

                    // Проверка на то, что такая книга уже есть
                    std::string checkBookSQL = "SELECT * FROM books WHERE title = '" +
book.title + "';";
                    char* errMsg = nullptr;
                    sqlite3_stmt* checkStmt;

                    int checkRC = sqlite3_prepare_v2(db, checkBookSQL.c_str(), -1,
&checkStmt, nullptr);

                    if (checkRC != SQLITE_OK) {
                        std::cerr << "Ошибка при подготовке SQL-запроса на проверку: "
<< sqlite3_errmsg(db) << std::endl;
                        sqlite3_finalize(checkStmt);
                        return;
                    }

                    checkRC = sqlite3_step(checkStmt);

                    if (checkRC == SQLITE_ROW) {
                        std::cout << "Книга '" << book.title << "' уже существует в
базе данных. Пожалуйста, введите другое название.\n";
                        sqlite3_finalize(checkStmt);
                    }
                    else {
                        isUnique = true;
                    }
                }
                while (true) {
                    std::cout << "Введите нового автора книги: ";
                    std::getline(std::cin, book.author);
                    if (isFirstLetterCapitalAndLatin(book.author)) {
                        break;
                    }
                    else {
                        std::cout << "Автор должен начинаться с заглавной буквы либо с
цифры." << std::endl;
                    }
                }

                do {
                    std::cout << "Введите новый дату выпуска (в формате xx.xx.xxxx): ";
                    std::getline(std::cin, book.data);
                    if (!isValidYear(book.data)) {
                        std::cout << "Неверный формат даты. Пожалуйста, введите дату в
формате xx.xx.xxxx.\n";
                    }
                } while (!isValidYear(book.data));

                while (true) {
                    std::cout << "Введите новый жанр книги: ";
                    std::getline(std::cin, book.genre);
                    if (isFirstLetterCapitalAndLatin(book.genre)) {
                        break;
```

```
                    }
                    else {
                        std::cout << "Жанр должен начинаться с заглавной буквы либо с
цифры." << std::endl;
                    }
                }

                std::string updateBookSQL = "UPDATE books SET title = '" + book.title +
"', author = '" +
                    book.author + "', data = '" + book.data + "', genre = '" +
book.genre + "' WHERE title = '" + bookTitle + "';";
                char* errMsg = nullptr;
                int rc = executeSQL(updateBookSQL, 0, 0, &errMsg);

                if (rc != SQLITE_OK) {
                    std::cerr << "Ошибка редактирования книги: " << errMsg <<
std::endl;
                    sqlite3_free(errMsg);
                }
                else {
                    std::cout << "Книга успешно отредактирована!\n";
                }
                return;
            }
            break;
            case 2:
            {
                std::string newBookTitle;
                bool isUnique = false;

                while (!isUnique) {
                    while (true) {
                        std::cout << "Введите новое название книги: ";
                        std::getline(std::cin, newBookTitle);
                        if (isFirstLetterCapitalAndLatin(newBookTitle)) {
                            break;
                        }
                        else {
                            std::cout << "Название книги должно начинаться с заглавной
буквы либо с цифры." << std::endl;
                        }
                    }

                    // Проверка на то, что такая книга уже есть
                    std::string checkBookSQL = "SELECT * FROM books WHERE title = '" +
newBookTitle + "';";
                    char* errMsg = nullptr;
                    sqlite3_stmt* checkStmt;

                    int checkRC = sqlite3_prepare_v2(db, checkBookSQL.c_str(), -1,
&checkStmt, nullptr);

                    if (checkRC != SQLITE_OK) {
                        std::cerr << "Ошибка при подготовке SQL-запроса на проверку: "
<< sqlite3_errmsg(db) << std::endl;
                        sqlite3_finalize(checkStmt);
                        return;
                    }

                    checkRC = sqlite3_step(checkStmt);
```

```
                    if (checkRC == SQLITE_ROW) {
                        std::cout << "Книга '" << newBookTitle << "' уже существует в
базе данных. Пожалуйста, введите другое название.\n";
                        sqlite3_finalize(checkStmt);
                    }
                    else {
                        isUnique = true;
                    }
                }

                // Обновляем название книги, выполняя соответствующий SQL-запрос
                std::string updateTitleSQL = "UPDATE books SET title = '" +
newBookTitle + "' WHERE title = '" + bookTitle + "';";
                char* updateErrMsg = nullptr;
                int rc = executeSQL(updateTitleSQL, 0, 0, &updateErrMsg);

                if (rc != SQLITE_OK) {
                    std::cerr << "Ошибка при выполнении обновления: " << updateErrMsg
<< std::endl;
                    sqlite3_free(updateErrMsg);
                }
                else {
                    std::cout << "Название книги успешно обновлено!\n";
                }
                return;
            }
            break;
            case 3:
            {
                Book book;
                while (true) {
                    std::cout << "Введите нового автора книги: ";
                    std::getline(std::cin, book.author);
                    if (isFirstLetterCapitalAndLatin(book.author)) {
                        break;
                    }
                    else {
                        std::cout << "Автор должен начинаться с заглавной буквы либо с
цифры." << std::endl;
                    }
                }

                std::string updateAuthorSQL = "UPDATE books SET author = '" +
book.author + "' WHERE title = '" + bookTitle + "';";
                char* errMsg = nullptr;
                int rc = executeSQL(updateAuthorSQL, 0, 0, &errMsg);

                if (rc != SQLITE_OK) {
                    std::cerr << "Ошибка редактирования автора книги: " << errMsg <<
std::endl;
                    sqlite3_free(errMsg);
                }
                else {
                    std::cout << "Автор книги успешно отредактирован!\n";
                }
            }
            break;
            case 4:
            {
```

```
                Book book;
                while (true) {
                    std::cout << "Введите новый жанр книги: ";
                    std::getline(std::cin, book.genre);
                    if (isFirstLetterCapitalAndLatin(book.genre)) {
                        break;
                    }
                    else {
                        std::cout << "Жанр должен начинаться с заглавной буквы либо с
цифры." << std::endl;
                    }
                }

                std::string updateGenreSQL = "UPDATE books SET genre = '" + book.genre
+ "' WHERE title = '" + bookTitle + "';";
                char* errMsg = nullptr;
                int rc = executeSQL(updateGenreSQL, 0, 0, &errMsg);

                if (rc != SQLITE_OK) {
                    std::cerr << "Ошибка редактирования жанра книги: " << errMsg <<
std::endl;
                    sqlite3_free(errMsg);
                }
                else {
                    std::cout << "Жанр книги успешно отредактирован!\n";
                }
            }
            break;
            case 5:
            {
                Book book;
                do {
                    std::cout << "Введите новую дату выпуска (в формате xx.xx.xxxx): ";
                    std::getline(std::cin, book.data);
                    if (!isValidYear(book.data)) {
                        std::cout << "Неверный формат даты. Пожалуйста, введите дату в
формате xx.xx.xxxx.\n";
                    }
                } while (!isValidYear(book.data));

                std::string updateYearSQL = "UPDATE books SET data = '" + book.data +
"' WHERE title = '" + bookTitle + "';";
                char* errMsg = nullptr;
                int rc = executeSQL(updateYearSQL, 0, 0, &errMsg);

                if (rc != SQLITE_OK) {
                    std::cerr << "Ошибка редактирования даты выпуска книги: " << errMsg
<< std::endl;
                    sqlite3_free(errMsg);
                }
                else {
                    std::cout << "Дата выпуска книги успешно отредактирована!\n";
                }
            }
            break;
            case 0:
                return;
            default:
                std::cout << "Такого числа в пункте меню нет.\n";
                break;
```

```
            }
        } while (true);
    }
}

void printBooksFromStatement(sqlite3_stmt* stmt) {
    int bookNumber = 1;
    bool resultsFound = false;
    int rc = sqlite3_step(stmt);

    while (rc == SQLITE_ROW) {
        if (bookNumber == 1) {
            std::cout << std::setw(20) << std::left << "Номер"
                << std::setw(23) << std::left << "Название"
                << std::setw(23) << std::left << "Автор"
                << std::setw(23) << std::left << "Дата"
                << std::setw(23) << std::left << "Жанр"
                << std::endl;
            std::cout << "-------------------------------------------------------------
----------------------------------------\n";
        }

        std::cout << std::setw(20) << std::left << bookNumber
            << std::setw(23) << std::left << sqlite3_column_text(stmt, 0)
            << std::setw(23) << std::left << sqlite3_column_text(stmt, 1)
            << std::setw(23) << std::left << sqlite3_column_text(stmt, 2)
            << std::setw(23) << std::left << sqlite3_column_text(stmt, 3)
            << std::endl;

        rc = sqlite3_step(stmt);
        bookNumber++;
        resultsFound = true;
    }

    if (!resultsFound) {
        std::cout << "Книги не найдены." << std::endl;
    }

    if (rc != SQLITE_DONE) {
        std::cerr << "Ошибка выполнения SQL-запроса: " << sqlite3_errmsg(db) <<
std::endl;
    }
}

void searchBooks(int searchOption) {
    std::cin.ignore();
    std::string searchCriteria;
    std::string searchSQL;

    switch (searchOption) {
    case 1:
        std::cout << "Введите название книги для поиска: ";
        std::getline(std::cin, searchCriteria);
        toUpper(searchCriteria);
        searchSQL = "SELECT title, author, data, genre FROM books WHERE title = '" +
searchCriteria + "';";
        break;
    case 2:
        std::cout << "Введите имя автора для поиска книг: ";
        std::getline(std::cin, searchCriteria);
```

```
        toUpper(searchCriteria);
        searchSQL = "SELECT title, author, data, genre FROM books WHERE author = '" +
searchCriteria + "';";
        break;
    case 3:
        std::cout << "Введите жанр книги для поиска: ";
        std::getline(std::cin, searchCriteria);
        toUpper(searchCriteria);
        searchSQL = "SELECT title, author, data, genre FROM books WHERE genre = '" +
searchCriteria + "';";
        break;
    default:
        std::cerr << "Некорректный вариант поиска." << std::endl;
        return;
    }

    char* errMsg = nullptr;
    sqlite3_stmt* stmt;

    int rc = sqlite3_prepare_v2(db, searchSQL.c_str(), -1, &stmt, nullptr);

    if (rc != SQLITE_OK) {
        std::cerr << "Ошибка подготовки SQL-запроса: " << sqlite3_errmsg(db) <<
std::endl;
        return;
    }

    printBooksFromStatement(stmt);
    sqlite3_finalize(stmt);
}

void printAllBooks() {
    std::string selectAllBooksSQL = "SELECT title, author, data, genre FROM books;";
    char* errMsg = nullptr;
    sqlite3_stmt* stmt;

    int rc = sqlite3_prepare_v2(db, selectAllBooksSQL.c_str(), -1, &stmt, nullptr);

    if (rc != SQLITE_OK) {
        std::cerr << "Ошибка подготовки SQL-запроса: " << sqlite3_errmsg(db) <<
std::endl;
        return;
    }

    printBooksFromStatement(stmt);

    sqlite3_finalize(stmt);
}

void clearDatabase() {
    const char* clearTableSQL = "DELETE FROM books;";
    char* errMsg = nullptr;
    int rc = executeSQL(clearTableSQL, 0, 0, &errMsg);

    if (rc != SQLITE_OK) {
        std::cerr << "Ошибка очистки базы данных: " << errMsg << std::endl;
        sqlite3_free(errMsg);
    }
    else {
        std::cout << "База данных успешно очищена." << std::endl;
```

```
    }
}

void printBooksSortedByAll(const std::string& criteria) {
    std::string selectSortedBooksSQL = "SELECT title, author, data, genre FROM books
ORDER BY " + criteria + ";";
    char* errMsg = nullptr;
    sqlite3_stmt* stmt;

    int rc = sqlite3_prepare_v2(db, selectSortedBooksSQL.c_str(), -1, &stmt, nullptr);

    if (rc != SQLITE_OK) {
        std::cerr << "Ошибка подготовки SQL-запроса: " << sqlite3_errmsg(db) <<
std::endl;
        return;
    }

    printBooksFromStatement(stmt);
    sqlite3_finalize(stmt);
}

int main() {
    system("chcp 1251 > nul");
    // Открытие базы данных
    int rc = sqlite3_open("library.db", &db);
    int choice = 0;
    if (rc) {
        std::cerr << "Ошибка открытия/создания базы данных: " << sqlite3_errmsg(db) <<
std::endl;
        return rc;
    }

    // Создание таблицы books, если она не существует
    const char* createTableSQL = "CREATE TABLE IF NOT EXISTS books ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT,"
        "title TEXT NOT NULL,"
        "author TEXT NOT NULL,"
        "data TEXT NOT NULL,"
        "genre TEXT NOT NULL);";
    char* errMsg = nullptr;
    rc = executeSQL(createTableSQL, 0, 0, &errMsg);

    if (rc != SQLITE_OK) {
        std::cerr << "Ошибка создания таблицы: " << errMsg << std::endl;
        sqlite3_free(errMsg);
        return rc;
    }

    do {
        std::cout << "Выберите операцию:\n"
            << "1. Поиск книги по названию\n"
            << "2. Поиск книги по автору\n"
            << "3. Поиск книги по жанру\n"
            << "4. Добавить книгу\n"
            << "5. Редактировать книгу\n"
            << "6. Удалить книгу\n"
            << "7. Вывести все книги из базы данных\n"
            << "8. Вывод отсортированных книг\n"
            << "9. Очистить базу данных\n"
            << "0. Выйти\n"
```

```
        << "Введите число: ";
    std::cin >> choice;
    while (std::cin.fail())
    {
        std::cin.clear();
        std::cin.ignore(10000, '\n');
        std::cout << "\n";
        std::cout << "Такого числа в пункте меню нет.\n";
        std::cout << "Выберите повторно: ";
        std::cin >> choice;
    }

    switch (choice) {
  case 1:
         std::cout << "\n";
         searchBooks(1);
  break;
  case 2:
         std::cout << "\n";
         searchBooks(2);
  break;
  case 3:
         std::cout << "\n";
         searchBooks(3);
  break;
    case 4:
        std::cout << "\n";
        addBook();
        break;
    case 5:
        std::cout << "\n";
        editBook();
        break;
    case 6:
        std::cout << "\n";
        deleteBook();
        break;
    case 7:
        std::cout << "\n";
        printAllBooks();
        break;
    case 8:
        int subChoice;
        do {
            std::cout << "\n1. Список книг отсортированных по названию\n"
                << "2. Список книг отсортированных по автору\n"
                << "3. Список книг отсортированных по дате выпуска\n"
                << "4. Список книг отсортированных по жанру\n"
                << "5. Выход в главное меню\n"
                << "Введите число: ";
            std::cin >> subChoice;
            while (std::cin.fail() || subChoice < 1 || subChoice > 5) {
                std::cin.clear();
                std::cin.ignore(10000, '\n');
                std::cout << "\nТакого числа в пункте меню нет.\n";
                std::cout << "Выберите повторно: ";
                std::cin >> subChoice;
            }

            switch (subChoice) {
```

```
case 1:
    std::cout << "\n";
    printBooksSortedByAll("title");
    break;
case 2:
    std::cout << "\n";
    printBooksSortedByAll("author");
    break;
case 3:
    std::cout << "\n";
    printBooksSortedByAll("SUBSTR(data, 7, 4) DESC");
    break;
case 4:
    std::cout << "\n";
    printBooksSortedByAll("genre");
    break;
                case 5:
                    break; // Выход в главное меню
                }
            } while (subChoice != 5);
            break;
        case 9:
            std::cout << "\n";
            clearDatabase();
            break;
        case 0:
            std::cout << "\n";
            std::cout << "Программа завершена.\n";
            break;
        default:
            std::cout << "\n";
            std::cout << "Такого числа в пункте меню нет.\n";
            break;
        }
    } while (choice != 0);

    sqlite3_close(db); // Закрываем базу данных при завершении программы
    return 0;
}
```